

Niks Blog CMS

Project Guide

How to run this project in Visual Studio Code
and how the application works

Stack: HTML | CSS | JavaScript (ES Modules)
Storage: Browser localStorage

1. Introduction

Niks Blog CMS is a modern blogging platform that runs entirely in the browser. It includes a public website for reading articles, user dashboards for readers and writers, and an administrator panel. No database server is required - all data is stored in localStorage.

2. System Requirements

- Windows 10/11, macOS, or Linux
- Modern browser: Chrome, Edge, or Firefox
- Visual Studio Code (recommended)
- Optional: Python 3 or Node.js for alternative local servers

3. Install Visual Studio Code

Step 1 - Download VS Code

Visit <https://code.visualstudio.com/> and download the installer for your operating system. Run the installer and enable 'Add to PATH' if offered.

Step 2 - Install Live Server extension

- Open VS Code
- Press Ctrl+Shift+X to open Extensions
- Search for 'Live Server' by Ritwick Dey
- Click Install
- A 'Go Live' button appears in the bottom status bar when ready

4. Open the Project in VS Code

- File -> Open Folder
- Select the 'chat app' project folder
- Confirm - the Explorer shows index.html, auth/, js/, css/, etc.

5. Run the Project

Method A - Live Server (Recommended)

- Right-click index.html in the Explorer
- Choose 'Open with Live Server'
- Browser opens at <http://127.0.0.1:5500/index.html>

Do NOT double-click HTML files. ES modules require http:// not file:// URLs.

Method B - Python

```
cd path/to/chat app
python -m http.server 8080
# Open http://localhost:8080
```

Method C - Node.js

```
npx serve .
```

```
# Open the URL shown in the terminal
```

6. Project Structure

```
chat app/  
  index.html      - Homepage  
  blog.html       - Explore articles  
  post.html       - Single article  
  auth/login.html - Login (user + admin)  
  auth/register.html  
  user/dashboard.html - User dashboard  
  admin/dashboard.html - Admin panel  
  js/state.js     - Data & authentication  
  js/common.js    - Shared UI logic  
  css/style.css   - Global styles
```

7. How It Works

Architecture: HTML pages load JavaScript ES modules. The state.js module manages all data in localStorage. When you read an article, post.js loads the post by ID from the URL, converts Markdown to HTML using marked.js, and highlights code with highlight.js.

Data flow

- First visit: sample posts and admin user are created in localStorage
- Login: credentials checked against users array in state.js
- Session: safe user object stored (password never kept in session)
- Posts: Markdown stored as text, rendered on post.html

8. User Roles

Regular User

Register at auth/register.html. Redirects to user/dashboard.html with reading history, bookmarks, and ability to write own stories.

Administrator

On login page, click 'Are you an admin?' and sign in with admin credentials. Opens admin/dashboard.html to manage all posts, users, and comments.

9. localStorage Keys

- niks_blog_data - all posts, users, categories
- niks_current_user - logged-in session
- niks_theme - dark or light mode
- niks_reading_{userId} - reading history
- niks_bookmarks_{userId} - saved articles

Reset: F12 -> Application -> Local Storage -> delete niks_* keys -> refresh.

10. Page Reference

- index.html + app.js - Homepage, featured posts, stats
- blog.html + blog-listing.js - Search, filters, sort
- post.html + post.js - Article, comments, bookmarks
- user/dashboard.html + user-dashboard.js - Personal hub
- admin/dashboard.html + admin.js - Platform management

11. Troubleshooting

- Blank page: use Live Server, not file://
- Modules error: serve over http://localhost
- Login fails: check browser console (F12)
- No posts: clear localStorage and refresh

12. Quick URL Reference (Live Server)

- Home: /index.html
- Explore: /blog.html
- Login: /auth/login.html
- Register: /auth/register.html
- User dashboard: /user/dashboard.html
- Admin panel: /admin/dashboard.html

For full documentation see README.md and docs/PROJECT_GUIDE.md in the project folder.